

Step 4 Results Note — Fine-Tuning RoBERTa for Masked-Word Reconstruction

Corpus construction, fine-tuning pipeline, debugging journey, and evaluation against the pretrained baseline

Problem ID	ML-T2-022
Candidate	Gyimah Ramsey Opoku
Student ID	9021623
Institution	KNUST, Kumasi, Ghana
Project Theme	Recovering Structure From Damaged or Poorly Scanned Documents
Stage	Stage 2 — Development, Step 4 of 5
Date	19 September 2026

1. Purpose

Step 3 established a pretrained-RoBERTa baseline for reconstructing missing words in damaged document text: 52.63% top-1 / 78.95% top-5 accuracy when the surrounding context was clean ground truth, and 47.37% top-1 / 78.95% top-5 when the surrounding context itself came from OCR on a damaged image. Both figures were measured on a single test sentence and were explicitly flagged as preliminary.

Step 4 asks whether fine-tuning RoBERTa on masked-word reconstruction, using a properly sized corpus with a genuinely held-out test set, improves on that pretrained baseline. This note documents the corpus and training pipeline built to answer that question, the sequence of bugs that surfaced while building it, how each was diagnosed and resolved, and the final, honest result of the comparison.

2. Method

2.1 Corpus construction

A public-domain corpus was assembled from Project Gutenberg rather than relying on a handful of hand-picked sentences. Four books were used as a training/validation pool (Pride and Prejudice, Frankenstein, The Adventures of Sherlock Holmes, Moby-Dick), and two entirely separate books were held out as a test set (Alice's Adventures in Wonderland, The Strange Case of Dr Jekyll and Mr Hyde). Holding out whole books, not just sentences, means the test set is never contaminated by the same author's or work's specific vocabulary patterns appearing during training.

Each book was downloaded in plain-text form, stripped of Gutenberg's licensing boilerplate, and split into sentences using a boundary regex. Sentences were filtered to a usable length (6–40 words), and any sentence containing digits, unusual characters, or standalone all-caps tokens of three or more letters was discarded — this last rule was added after an early pass showed chapter headings such as "CHAPTER XII" leaking into the sentence pool as if they were normal prose.

Split	Sentences	Source
Train	8,000	4-book pool
Validation	1,000	4-book pool
Held-out test	739	Alice in Wonderland + Jekyll & Hyde (combined, unseen books)

Table 1. Final corpus split sizes, by whole book to prevent train/test leakage.

2.2 Masked example generation

From each sentence, two masked training examples were generated by selecting two distinct word positions at random and masking one word per example — rather than one masked example per sentence — so the same sentence contributes more than one learning signal without exhausting it. Each masked word was tagged as a content word or a function word using a stopword list, so that reconstruction accuracy could later be broken down by word type; content words are the harder and more practically important case, since they carry the meaning a damaged document actually needs recovered.

Split	Examples	Content words	Function words
Train	16,000	8,486	7,514
Validation	2,000	1,087	913
Held-out test	1,478	777	701

Table 2. Masked-example counts, split by whether the masked word was a content word or a function word.

2.3 Fine-tuning setup

Training was run on a Kaggle Notebook with a T4 GPU, using HuggingFace's Trainer on top of roberta-base. Rather than relying on the library's default random masking (`DataCollatorForLanguageModeling`), each example's mask position and label were constructed explicitly: the target word was tokenized to find how many RoBERTa subword pieces it needs, that many `<mask>` tokens were inserted at the correct position, and only those positions were given real labels (all others set to -100, meaning "ignore for loss"). This makes the training signal exactly match the task the model will be evaluated on, rather than a generic random-masking objective.

Training ran for 3 epochs, batch size 16, learning rate 2e-5, with evaluation and checkpointing at the end of every epoch.

3. The Debugging Journey

Three distinct problems had to be found and fixed before this pipeline produced a trustworthy result. Each is recorded here because each changed the conclusion at the time it was found, and the final result only means what it means once all three are accounted for.

3.1 Zero loss from a misconfigured data collator

The first training run used HuggingFace's built-in masking collator with a masking probability parameter left at 0.0, on the reasoning that masking was already done manually in the data. In practice this meant the collator generated no labels at all, so the model received no training signal — loss stayed at zero or NaN throughout. This was fixed by removing the collator entirely and constructing the labels tensor directly inside the tokenization step, as described in 2.3.

3.2 The single-token shortcut, and why it was rejected

With labels constructed directly, an early version of the tokenization code only kept examples where the target word tokenized to exactly one RoBERTa subword. This was meant to sidestep the complexity of multi-token targets, but it discarded 96.8% of the training set (518 of 16,000 examples survived), and it disproportionately discarded content words — precisely the harder, more important case for this project.

This shortcut, along with a proposal to compensate by growing the corpus to 500,000 examples, was rejected on the grounds that it would deliberately exclude the words most in need of reconstruction, and that the premise was wrong: RoBERTa can be trained with multiple adjacent mask positions in one sequence without confusing which prediction belongs to which position, since each position's loss is computed independently against its own label.

The correct fix, implemented instead, was to keep every example, insert the exact number of <mask> tokens the target word needs, and label each masked position with its corresponding subword id. This raised the kept-example count from 518/16,000 to roughly 11,550/16,000 (the remainder being cases where the tokenized mask span did not align cleanly and were safely dropped, verified rather than assumed).

3.3 Evaluation-methodology mismatch, and the real root cause underneath it

The first evaluation of the fine-tuned model, using HuggingFace's single-mask fill-mask pipeline, showed catastrophic collapse: 3.17–3.92% top-1 accuracy overall and 0.00% on content words. Rather than accept this as a training failure, it was checked against the model's actual training objective: computing MLM loss directly (feeding the same input_ids, attention_mask, and labels used in training back through the model and reading outputs.loss). That comparison told a different story — pretrained RoBERTa scored 4.62–4.69 loss, the fine-tuned model scored 2.27–2.33, a clear improvement on the objective it was actually trained on. The fill-mask pipeline's poor showing was therefore a measurement artifact, not a training failure: a single-mask pipeline call cannot correctly score a word that needs several mask tokens, so most of the harder (content-word) targets were being evaluated by a method that could not represent them.

A corrected evaluation was then built: an iterative decoder that inserts the same number of mask tokens the target needs, predicts them one at a time (feeding each prediction back in before predicting the next), and scores the full reconstructed word as a whole. Running this on a freshly loaded, completely untouched pretrained model still produced near-zero accuracy (0.28–0.45% top-1) — a signal that could not be a fine-tuning bug, since no fine-tuning had touched that model. This ruled out the labels and the training data, and pointed at the tokenizer itself.

Direct inspection confirmed it: tokenizing the word "CHAPTER" returned seven tokens decoding to "C H A P T E R" — letter by letter — and ordinary words such as "the", "with", and "do" were similarly shredded into character fragments instead of the single subword tokens RoBERTa expects. The loaded class name, tokenization_roberta_old.RobertaTokenizerFast, gave the cause away: an incompatibility between transformers 5.0.0 and tokenizers 0.22.2 was silently falling back to a broken legacy tokenizer path. Forcing a fresh model download did not fix it, ruling out cache corruption. Pinning transformers==4.44.2 and tokenizers==0.19.1 and restarting the kernel fixed it outright — tokenizing " the" then correctly returned the single expected token.

4. Results

4.1 MLM loss (training-objective-consistent check)

Model	Average MLM loss on validation set
Pretrained RoBERTa (untouched)	4.62 – 4.69
Fine-tuned RoBERTa	2.27 – 2.33

Table 3. Loss computed on the exact objective the model was trained on. Lower is better.

4.2 Whole-word reconstruction accuracy (iterative decode, held-out test books)

Model	Overall top-1	Content-word top-1
Pretrained RoBERTa	48.31%	27.03%
Fine-tuned RoBERTa	48.51%	27.28%

Table 4. Whole-word top-1 accuracy on the held-out test set (*Alice in Wonderland* + *Jekyll & Hyde*), after the tokenizer fix, using the corrected iterative-decode evaluation described in 3.3.

5. Interpretation

Two results point in different directions, and both are kept rather than reconciled away. On MLM loss — the metric that matches exactly what the model was optimized for — fine-tuning produced a clear, large improvement (roughly halving loss). On whole-word reconstruction accuracy against the held-out test books, the improvement all but disappears: 48.31% → 48.51% overall, 27.03% → 27.28% on content words specifically. Neither gain is large enough to call fine-tuning a clear win for the practical task.

The most likely explanation is overfitting after the first epoch. Training loss fell every epoch (2.75 → 2.05 → 1.65), but validation loss rose after epoch 1 (2.62 → 2.63 → 2.69). The checkpoint saved at epoch 3 — the one used for the Table 4 comparison — may already be fitting patterns specific to the training books rather than genuinely improving reconstruction ability on unseen text. This is a legitimate negative result, not a broken pipeline: the model trains correctly (Table 3 shows it), but the specific checkpoint evaluated has likely been trained past its useful point.

It is also worth noting, honestly, that pretrained RoBERTa's content-word accuracy (27%) remains well below its overall accuracy (48%) both before and after fine-tuning. Reconstructing the semantically load-bearing words in a damaged sentence is the harder half of this task, and fine-tuning as run here did not close that gap.

6. Next Steps

- Re-run training with `load_best_model_at_end=True` (selecting on validation loss) so the checkpoint actually evaluated is the one before validation loss started rising, rather than the final epoch.
- Re-run the Table 4 comparison on that checkpoint to see whether the loss-based improvement in Table 3 translates into a real accuracy gain once overfitting is controlled for.
- Extend evaluation from single held-out sentences to the full 739-sentence / 1,478-example held-out test set, so the next accuracy figures are no longer preliminary in the way Step 3's were.
- Revisit the confidence-correctness relationship flagged in Step 3 (higher model confidence correlating with correct predictions) now that a larger test set exists to check it properly.
- Proceed toward Step 5 (confidence-based uncertainty flagging on top of reconstruction output) once the overfitting question above is resolved, per the Stage 2 Development Brief's five-step plan.